BITS PILANI WILP

Scalable Services (SEZG583) • M.Tech in Software Systems

Assignment 1: Comprehensive Deliverables Package

Microservices Architecture • Inventory Service • Concurrency Control (Challenge 1) • Business Invariants (BR1 & BR2)

Student Name:	Dr. Sunanda Pandita	**Assigned Role:**	Owner P1 (Inventory Service Lead)
BITS Student ID:	2025tm93132	**Challenge Owned:**	Challenge 1: Concurrency Control
Institutional Email:	2025tm93132@wilp.bits-pilani.ac.in	**Shared Output Lead:**	docs/business-rules.md (BR1–BR6)
GitHub Account:	@DrSunandaPandita	**Remote Repo:**	2022MT93122/ecommerce-scalable-services
Feature Branch:	feature/inventory-service	**Pull Request:**	PR #1 (Fully Tested & Pushed)

1. Executive Summary & Deliverables Matrix

This document establishes the verified deliverables for **Dr. Sunanda Pandita** in accordance with the **SEZG583 Assignment 1 Handbook**. The project implements a four-microservice distributed architecture governed by the database-per-service pattern. As Owner P1, Dr. Sunanda Pandita engineered the complete **Inventory Microservice**, enforced **Business Rule 1** (Stock Never Negative) and **Business Rule 2** (Confirmed Hold Protection), authored the 20-thread concurrency harness for **Challenge 1**, and captured empirical proof verifying zero overselling.

Deliverable Component	Specification Requirement	Author / Owner	Repository Location & Evidence Artifact	Status
Inventory Microservice	FastAPI standalone service, Dockerfile, isolated PostgreSQL DB, REST endpoints	Dr. Sunanda Pandita	`services/inventory/{app/, tests/, Dockerfile, requirements.txt}`	VERIFIED
Challenge 1 Concurrency Harness	20 parallel threads competing for 5 stock units; assert 5 pass, 15 fail, stock = 0	Dr. Sunanda Pandita	`scripts/concurrency.py`	VERIFIED
Business Rules Invariant Tests	Unit tests for BR1 (Atomic stock hold) and BR2	Dr. Sunanda Pandita	`services/inventory/tests/test_business_rules.py` (9/9 passed)	VERIFIED

Deliverable Component	Specification Requirement	Author / Owner	Repository Location & Evidence Artifact	Status
	(Release lock guard)			
Challenge 1 Evidence Folder	Execution log with timestamps, SQL database dump, architectural explanation	Dr. Sunanda Pandita	`evidence/challenge-1-concurrency/{run-log.txt, sql-proof.txt, README.md}`	VERIFIED
Business Rules Document	Formalization of BR1 through BR6 across multi-service boundaries	Dr. Sunanda Pandita (Lead)	`docs/business-rules.md`	VERIFIED
Inventory ER & Schema Docs	Entity-Relationship diagram, database constraints, hold state machine	Dr. Sunanda Pandita	`docs/inventory-erd.md`	VERIFIED
Personal Student Reflection	Half-page personal reflection: what built, hardest problem, future improvements	Dr. Sunanda Pandita	`docs/reflections/sunanda_pandita_reflection.md`	VERIFIED
AI Assistance Register	Transparent declaration of AI tools, tasks assisted, and human validations	Dr. Sunanda Pandita	`docs/AI_USE_LOG.md`	VERIFIED

2. Inventory Microservice Architecture & Data Design

The Inventory Service runs on port **8001** and connects exclusively to **`inventory_db`**. In conformance with distributed database boundaries, cross-service database access and foreign keys to external tables (e.g. orders) are strictly prohibited.

Entity-Relationship Schema

Table Name	Column	Data Type	Constraints	Description
products	`id`	INTEGER	PRIMARY KEY, AUTOINCREMENT	Surrogate primary key for product catalog items
	`name`	VARCHAR(120)	NOT NULL	Product title / model designation

Table Name	Column	Data Type	Constraints	Description
	`price_cents`	INTEGER	NOT NULL, CHECK (price_cents > 0)	Price in INR paise (e.g., 200,000 paise = Rs 2,000.00)
	`available`	INTEGER	NOT NULL, CHECK (available >= 0)	Available units for sale. Database constraint prevents oversell.
holds (reservations)	`id`	INTEGER	PRIMARY KEY, AUTOINCREMENT	Surrogate primary key for inventory reservations
	`order_id`	VARCHAR(64)	UNIQUE, INDEX, NOT NULL	Correlation ID from Order Service; enforces idempotency
	`product_id`	INTEGER	FOREIGN KEY (products.id), NOT NULL	Local reference to the catalog product
	`quantity`	INTEGER	NOT NULL, CHECK (quantity > 0)	Reserved unit quantity
	`status`	VARCHAR(20)	NOT NULL, DEFAULT 'HELD'	Lifecycle state: HELD • CONFIRMED • RELEASED

3. Business Rules Enforcement (Lead: Dr. Sunanda Pandita)

BR1: Stock Never Negative (Atomic Conditional Reservation)

Problem: Under concurrent bursts (flash sales), multi-threaded application logic executing a check before an update (`SELECT available ... IF available >= qty THEN UPDATE`) creates a critical race window. Interleaved execution allows multiple threads to read the same remaining stock and over-decrement into negative balances.

Enforcement: We eliminate application-level race conditions by shifting the evaluation to an atomic conditional SQL update executed inside an engine row-level lock:

```
UPDATE products SET available = available - :quantity WHERE id = :product_id AND available >=
:quantity RETURNING id, available;
```

If zero rows match the condition, the transaction rolls back immediately and returns HTTP `409 CONFLICT (OUT_OF_STOCK)` . Furthermore, the database table enforces an immutable `CHECK (available >= 0)` constraint.

BR2: Confirmed Hold Cannot Be Released (State-Machine Guard)

Problem: Once payment has succeeded and an inventory hold transitions to `CONFIRMED` , late timeout messages or erroneous cancellation signals must never release purchased stock back to available warehouse inventory.

Enforcement: When `POST /api/v1/stock/{id}/release` is invoked, the service locks the hold row:
• If `status == "CONFIRMED"` : Throws `ValueError("Confirmed stock cannot be released")` mapping to HTTP 409.
• If `status == "RELEASED"` : Returns HTTP 200 idempotently without incrementing stock again.
• If `status == "HELD"` : Increments `products.available` , transitions status to `RELEASED` , and commits.

4. REST API Endpoint Reference (Port 8001)

HTTP Method	Endpoint Route	Purpose & Request Payload	Expected Responses
GET	`/health`	Operational readiness and database connectivity probe	`200 OK {"status": "ok", "service": "inventory"}`
POST	`/api/v1/products`	Create product: `{"name": str, "price_cents": int, "available": int}`	`201 Created {"id": 1, ...}`
GET	`/api/v1/products`	List catalog products and available stock count	`200 OK [{"id": 1, "available": 5, ...}]`
GET	`/api/v1/products/{id}`	Fetch specific product details and live availability	`200 OK` or `404 Not Found`
POST	`/api/v1/stock/reserve`	Atomic hold: `{"order_id": str, "product_id": int, "quantity": int}`	`201 Created {"hold_id": int, "status": "HELD"}` `409 Conflict (OUT_OF_STOCK)`
POST	`/api/v1/stock/{id}/confirm`	Confirm hold upon successful payment completion	`200 OK {"status": "CONFIRMED"}` `409 Conflict (Illegal transition)`

HTTP Method	Endpoint Route	Purpose & Request Payload	Expected Responses
POST	`/api/v1/stock/{id}/release`	Release hold on payment cancellation / timeout compensation	`200 OK {"status": "RELEASED"}` `409 Conflict (BR2 Violation)`

5. Challenge 1: Concurrency Control Execution Results

The test harness (`scripts/concurrency.py`) initializes a test product with **5 available units** and spawns **20 concurrent threads** using Python's `concurrent.futures.ThreadPoolExecutor` . Each thread attempts to reserve 1 unit simultaneously using unique order IDs.

```
====================================================================== CHALLENGE 1: CONCURRENT STOCK
RESERVATION TEST Target Service: http://localhost:8001
====================================================================== [*] Health Check: HTTP 200 ->
{'status': 'ok', 'service': 'inventory'} [+] Created Test Product ID: 1 | Initial Stock: 5 units |
Price: INR 2000.00 [*] Dispatching 20 concurrent reservation requests (1 unit each)... ----------------
---------------------------------------------------- Request #01 | Order: ord-burst-01-b5c233ea |
HTTP 201 (SUCCESS) in 36.1ms Request #03 | Order: ord-burst-03-2ba9bec2 | HTTP 201 (SUCCESS) in 42.2ms
Request #02 | Order: ord-burst-02-c7070869 | HTTP 201 (SUCCESS) in 54.9ms Request #04 | Order: ord-
burst-04-814fc84b | HTTP 201 (SUCCESS) in 59.6ms Request #05 | Order: ord-burst-05-613b7c63 | HTTP 201
(SUCCESS) in 61.0ms Request #06 | Order: ord-burst-06-c7e1efab | HTTP 409 (REJECTED (OUT_OF_STOCK)) in
58.1ms Request #07 | Order: ord-burst-07-dacb22f4 | HTTP 409 (REJECTED (OUT_OF_STOCK)) in 59.3ms ...
[13 additional controlled 409 rejections omitted for brevity] ...
====================================================================== Total Requests Dispatched : 20
Successful Reservations : 5 (Expected: 5) Rejected Reservations : 15 (Expected: 15) Initial Product
Stock : 5 Final Available Stock : 0 (Expected: 0) [✓] PASS: All concurrency invariants verified! [✓]
BR1 Enforced: Stock never dropped below zero (No overselling).
======================================================================
```

6. Post-Test SQL Database State Proof

Querying the PostgreSQL database directly confirms that exactly 5 holds were committed and available stock reached exactly 0 without overselling:

```
$ docker compose exec -T postgres psql -U inventory_user -d inventory_db -c "SELECT id, name, price_cents, available FROM products WHERE id = 1;" id | name | price_cents | available ----+---------- ------------------------+-------------+----------- 1 | Mechanical Keyboard (Batch-20542d) | 200000 | 0 (1 row) $ docker compose exec -T postgres psql -U inventory_user -d inventory_db -c "SELECT id, order_id, quantity, status FROM holds WHERE product_id = 1;" id | order_id | quantity | status ----+--- ------------------+----------+-------- 1 | ord-burst-01-b5c233ea | 1 | HELD 2 | ord-burst-03-2ba9bec2 | 1 | HELD 3 | ord-burst-02-c7070869 | 1 | HELD 4 | ord-burst-04-814fc84b | 1 | HELD 5 | ord-burst-05-613b7c63 | 1 | HELD (5 rows)
```

7. Automated Pytest Test Suite Results

Automated unit tests in `services/inventory/tests/test_business_rules.py` validate all core business invariants:

```
============================= test session starts ============================== platform darwin -- Python 3.14.7, pytest-9.1.1, pluggy-1.7.0 rootdir: /services/inventory, configfile: pytest.ini collected 9 items tests/test_business_rules.py::test_health_check PASSED [ 11%] tests/test_business_rules.py::test_create_and_get_product PASSED [ 22%] tests/test_business_rules.py::test_br1_atomic_reserve_prevents_oversell PASSED [ 33%] tests/test_business_rules.py::test_reserve_idempotency_same_order PASSED [ 44%] tests/test_business_rules.py::test_reserve_conflict_different_payload PASSED [ 55%] tests/test_business_rules.py::test_br2_confirmed_hold_cannot_be_released PASSED [ 66%] tests/test_business_rules.py::test_release_held_stock_restores_inventory PASSED [ 77%] tests/test_business_rules.py::test_released_hold_cannot_be_confirmed PASSED [ 88%] tests/test_contract.py::test_service_file_exists PASSED [100%] ============================== 9 passed in 0.30s ===============================
```

8. Demo Video Storyboard & Script (Minutes 0:00 – 3:00)

Time Window	Visual Screen Action	Spoken Script (Dr. Sunanda Pandita)
0:00 – 0:45	Show Docker Desktop GUI with 5 running containers (Postgres, Inventory, Order, Payment, Notification).	*"Hello everyone. I am Dr. Sunanda Pandita. For our Scalable Services team project, I led the Inventory Service and Challenge 1 on Concurrency Control. As you see on the screen in Docker Desktop, our system follows the database-per-service microservice pattern with four isolated REST services and PostgreSQL. No service has permission to access another database directly."*
0:45 – 1:30	Open browser to `http://localhost:8001/docs` (Swagger UI). Execute `GET /health` and `GET /api/v1/products`.	*"Here is the interactive OpenAPI interface for the Inventory Service running on port 8001. All operations are versioned under /api/v1/, prices are stored as integer paise to prevent floating-point rounding errors, and our /health endpoint confirms database operability. Next, I will demonstrate how stock is reserved atomically."*
1:30 – 2:30	Open terminal and execute: `python scripts/concurrency.py`	*"Now for Challenge 1: Concurrency. We seed a Mechanical Keyboard with only 5 available units. Using Python's ThreadPoolExecutor, we blast the service with 20 parallel reservation requests at the same millisecond. As shown in the live terminal logs: exactly 5 requests succeed with HTTP*

Time Window	Visual Screen Action	Spoken Script (Dr. Sunanda Pandita)
		201 Created, while 15 requests receive a controlled HTTP 409 OUT_OF_STOCK rejection."
2:30 – 3:00	Show SQL query output showing `available = 0` and exactly 5 rows in `holds`.	*"Querying PostgreSQL directly proves that our single atomic conditional UPDATE prevented race conditions completely. Stock is exactly 0 with zero overselling, enforcing Business Rule 1. Furthermore, our unit tests verify Business Rule 2: confirmed holds cannot be released. I will now hand over to Tarun to demonstrate Order orchestration and idempotency."*

9. Report Appendix Items

A. Team Contribution Table

Student Name	Student ID	Institutional Email	Service Owned	Challenge Owned	Deliverables & Verified Evidence
Dr. Sunanda Pandita	2025tm93132	2025tm93132@wilp.bits-pilani.ac.in	**Inventory Service**	**Challenge 1: Concurrency**	Inventory REST APIs, atomic conditional stock update, hold state machine (BR1 & BR2), unit test suite (9 tests), scripts/concurrency.py, evidence/challenge-1-concurrency/

B. Individual Reflection: Dr. Sunanda Pandita

1. What I Built: As the Inventory Service owner, I designed and implemented the entire stock catalog and inventory hold lifecycle management component. I structured the service using FastAPI and SQLAlchemy 2.0 with a dedicated database schema (`products` and `holds` tables) adhering strictly to the database-per-service microservice pattern. I authored the atomic conditional stock decrement query, implemented the hold state machine with release guards (BR2), exposed standardized OpenAPI documentation with correlation ID middleware, wrote comprehensive automated unit tests, and created the 20-thread concurrency validation harness.

2. Hardest Problem Solved: The primary engineering challenge was preventing race conditions under high-throughput concurrent requests without degrading service responsiveness. Initially, separating the availability check from the update in application logic (`SELECT` followed by `UPDATE`) caused severe overselling when 20 parallel threads competed for the last 5 keyboard units. When exploring coarse table-level locking, concurrency throughput plummeted. I resolved this by enforcing an atomic conditional `UPDATE ... WHERE available >= :qty RETURNING id, available` inside a single database transaction. This shifted the locking boundary directly to the row-level update step in the database engine, returning an immediate 409 `OUT_OF_STOCK` if zero rows matched.

3. What I Would Change (Future Improvements): For production scaling in Assignment 2, I would incorporate: (1) An in-memory Redis cache with atomic Lua scripts to offload read-heavy stock lookups; (2) An automated time-bound hold expiration worker to release held inventory if payment is not finalized within 15 minutes; and (3) Asynchronous message-broker integration (RabbitMQ/Kafka) to decouple inventory compensation from synchronous Order Service HTTP calls.

C. AI Assistance Disclosure

Date	Member	Tool Used	Task Assisted	Generated Aid	Human Review, Verification & Modifications
2026-10-25	Dr. Sunanda Pandita	Antigravity AI	Project planning & handbook analysis	Scaffold layout, architecture notes	Refined service boundaries and isolation rules for database-per-service pattern.
2026-10-26	Dr. Sunanda Pandita	Antigravity AI	Atomic stock decrement query & hold lifecycle	Conditional SQL update & state transitions	Adapted to SQLAlchemy 2.0 with PostgreSQL/SQLite compatibility, added custom idempotency duplicate check. Tested with pytest.
2026-10-26	Dr. Sunanda	Antigravity AI	Concurrency test harness & SQL	Multi-threaded test script	Verified against 20 concurrent requests for 5 stock units, confirmed 5 accepted

Date	Member	Tool Used	Task Assisted	Generated Aid	Human Review, Verification & Modifications
	Pandita		proof		and 15 rejected with 0 final stock.